
Policy-Value Learning for Magic: The Gathering Arena Cube Draft

Anonymous Authors¹

Abstract

We develop and evaluate a learning system for Magic: The Gathering Arena cube draft recommendations from public 17lands data. The method separates the problem into a pick policy trained from human draft logs and outcome/value signals trained from game results. At inference, the policy supplies calibrated in-pack priors while value-derived terms rerank or fine-tune recommendations. The architecture is cube-list-conditioned, represents cards by learned and Scryfall-derived features, and uses only information observable to the drafting player: current pack, own pool, cards seen but not picked, pack index, pick index, and known cube list. On a held-out Powered Cube split, the strongest human-imitation model obtains top-1 accuracy 0.624 and top-3 accuracy 0.909. We further train game-data preference policies from learned card bonuses. A safer game-data policy improves generated preference accuracy from 0.549 to 0.641 while preserving top-1 0.616; an aggressive preference-accuracy policy reaches preference accuracy 0.726 but drops top-1 human agreement to 0.509. We also implement an experimental MTGA Player.log follower that reconstructs quick-draft pack and pick state from live client logs. Finally, we add an inference-time Monte Carlo search layer; with 25 simulations it gives live search-based reranking at roughly one-second median CPU latency and qualitatively produces more lane-consistent draft traces. For live cube updates, we handle cards absent from the training data and learned embedding vocabulary by precomputing function-aware nearest-neighbor pseudo-embeddings from Scryfall rules text and curated role tags.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Problem

A cube draft consists of $S = 8$ seats, $P = 3$ packs per seat, and $K = 15$ cards per pack. At each decision t , the acting player observes a state

$$s_t = (x_t, y_t, z_t, m, p_t, k_t), \quad (1)$$

where $x_t \in \{0, 1\}^C$ is the current pack mask over a cube-local vocabulary of size C , $y_t \in \mathbb{N}^C$ is the player’s drafted pool, $z_t \in \mathbb{N}^C$ counts cards previously seen by the player but not selected, $m \in \{0, 1\}^C$ is the known cube-list mask, and (p_t, k_t) are pack and pick indices. The action set is the current pack,

$$\mathcal{A}(s_t) = \{i : x_{t,i} = 1\}. \quad (2)$$

The objective is to recommend a pick $a_t \in \mathcal{A}(s_t)$ that maximizes expected downstream deck performance, while remaining robust to partial cube-list knowledge and unseen cards.

2. Data

We use three public 17lands tables for a target format such as Cube - Powered: draft data, game data, and replay data (17Lands, 2026). Draft rows contain one human pick per row: the selected card, current pack card indicators, draft identifier, pack number, pick number, rank, and event metadata. Game rows contain one played game per row: draft identifier, build index, deck and sideboard card indicators, colors, rank, play/draw, mulligans, opponent metadata, and win/loss. Replay rows contain turn-level play traces and are deferred to future play-skill modeling.

The core supervised datasets are:

$$\mathcal{D}_\pi = \{(s_t, a_t^{\text{human}})\}_{t=1}^N, \quad (3)$$

$$\mathcal{D}_V = \{(d_j, b_j, c_j, r_j, w_j)\}_{j=1}^M, \quad (4)$$

where d_j is a main-deck vector, b_j is a sideboard vector, c_j are color features, r_j are rank/event covariates, and $w_j \in \{0, 1\}$ is the game outcome. Game rows are also aggregated by (draft_id, build_index) into empirical deck win rates.

3. Card Representation

Each card i has a learned global embedding and a feature-derived embedding:

$$e_i = E_i + \alpha g(f_i), \quad (5)$$

where $E_i \in \mathbb{R}^d$ is a learned embedding, f_i contains Scryfall-derived features (Scryfall, 2026) such as mana value, colors, type bits, rarity, power/toughness, keywords, and oracle-text embeddings, and g is an MLP. The scalar α is annealed from 1 to a positive floor, e.g. 0.3, so that known cards may specialize while unseen cards retain meaningful feature-based representations.

3.1. Function-aware kNN for unseen cards

Live Arena cube lists change, so a pack can contain a card absent from the policy’s learned embedding table. A naive fallback would use a shared UNK embedding, but this erases important distinctions between, for example, a new protection creature and a new removal spell. Instead, for an unseen card u we synthesize a learned-embedding proxy from nearest known cards:

$$\hat{E}_u = \sum_{j \in \mathcal{N}_k(u)} w_j E_j, \quad e_u = \hat{E}_u + \alpha g(f_u). \quad (6)$$

Neighbors are selected from the trained cube/scoring vocabulary, not the entire Scryfall database, to avoid obscure non-cube cards dominating the interpolation. Similarity combines three normalized channels: structured card features, face-balanced hashed rules text, and explicit functional role tags. We exclude card names from text similarity to avoid name leakage; for split, adventure, and modal cards, each face is embedded separately and averaged so one verbose half does not dominate the other. The role tags encode gameplay functions such as protection for another creature, equipment/aura tutoring, removal, counterspells, rituals, artifact synergies, graveyard tutoring, and fixing. In the live assistant, these neighbors are precomputed for the published cube list and loaded from disk, so inference uses only a small weighted average of cached known-card embeddings.

4. Pick Policy Model

The pick policy is a cube-context contextual preference ranker. Given card representations $\{e_i\}_{i=1}^C$, we encode the pool, seen-but-unpicked cards, and cube list with DeepSets:

$$h_y = \rho_y \left(\frac{\sum_i y_i \phi_y(e_i)}{\max(1, \sum_i y_i)} \right), \quad (7)$$

$$h_z = \rho_z \left(\frac{\sum_i z_i \phi_z(e_i)}{\max(1, \sum_i z_i)} \right), \quad (8)$$

$$h_m = \rho_m \left(\frac{\sum_i m_i \phi_m(e_i)}{\max(1, \sum_i m_i)} \right). \quad (9)$$

We also compute a color-commitment vector

$$q = \frac{\sum_i y_i \text{color}(i)}{\max(1, \sum_i y_i)} \in \mathbb{R}^5. \quad (10)$$

The context vector is

$$h(s) = \text{MLP}_\pi([h_y, h_z, h_m, q, u_p, v_k]), \quad (11)$$

where u_p and v_k are learned pack-index and pick-index embeddings. Card logits are

$$\ell_i(s) = h(s)^\top e_i + b(e_i), \quad (12)$$

with b a scalar bias head. The masked policy is

$$\pi_\theta(i | s) = \frac{\exp \ell_i(s)}{\sum_{j \in \mathcal{A}(s)} \exp \ell_j(s)}, \quad i \in \mathcal{A}(s). \quad (13)$$

4.1. Policy Loss

The default loss is masked cross-entropy on human picks:

$$\mathcal{L}_{\text{CE}}(\theta) = -\mathbb{E}_{(s,a) \sim \mathcal{D}_\pi} \log \pi_\theta(a | s). \quad (14)$$

We also use a contextual preference ranking loss over unpicked pack cards:

$$\mathcal{L}_{\text{CPR}}(\theta) = \mathbb{E}_{(s,a)} \frac{1}{|\mathcal{A}(s)| - 1} \sum_{j \in \mathcal{A}(s) \setminus \{a\}} [\gamma - \ell_a(s) + \ell_j(s)]_+. \quad (15)$$

The combined pick-imitation loss is

$$\mathcal{L}_\pi = \mathcal{L}_{\text{CE}} + \beta_{\text{CPR}} \mathcal{L}_{\text{CPR}}. \quad (16)$$

5. Outcome Value Model

The value model predicts game win probability from a built deck:

$$V_\psi(d, b, c, r) \in (0, 1). \quad (17)$$

Main deck and sideboard are encoded by DeepSets over the same card representations:

$$h_d = \rho_d \left(\frac{\sum_i d_i \phi_d(e_i)}{\max(1, \sum_i d_i)} \right), \quad (18)$$

$$h_b = \rho_b \left(\frac{\sum_i b_i \phi_b(e_i)}{\max(1, \sum_i b_i)} \right), \quad (19)$$

$$V_\psi(d, b, c, r) = \sigma(\text{MLP}_V([h_d, h_b, c, r])). \quad (20)$$

For per-game training the loss is binary cross-entropy:

$$\mathcal{L}_V(\psi) = -\mathbb{E}_{(d,b,c,r,w) \sim \mathcal{D}_V} [w \log V_\psi + (1 - w) \log(1 - V_\psi)]. \quad (21)$$

For deck-aggregate training, if a deck has n games and empirical win rate $\hat{p}$, we use the binomial negative log-likelihood

$$\mathcal{L}_V^{\text{agg}} = -\mathbb{E} [n \hat{p} \log V_\psi + n(1 - \hat{p}) \log(1 - V_\psi)]. \quad (22)$$

Rank, event type, and play/draw covariates can be included or ablated. For deployment scoring, we report both rank-conditioned predictions and rank-marginalized predictions.

Algorithm 1 One-step policy-value reranking

Require: State $s = (x, y, z, m, p, k)$, policy π_θ , value V_ψ , deckbuilder B , weight λ

- 1: **for** $a \in \mathcal{A}(s)$ **do**
- 2: $y^a \leftarrow y + \mathbf{1}_a$
- 3: $(d^a, b^a, c^a) \leftarrow B(y^a, m)$
- 4: $S(a) \leftarrow \log \pi_\theta(a | s) + \lambda \text{logit}(V_\psi(d^a, b^a, c^a, r_0))$
- 5: **end for**
- 6: **return** $\arg \max_a S(a)$

6. Pool-to-Deck Builder

Because V_ψ scores built decks, not raw draft pools, a deckbuilder B maps a drafted pool y and cube context m to a main deck and sideboard:

$$B(y, m) = (d, b, c). \quad (23)$$

The minimum viable B is a constrained optimizer that selects approximately 23 nonland spells plus lands, penalizes excessive colors, enforces curve constraints, and rewards fixing. A learned deckbuilder may later be trained from 17lands build rows. In algorithms below, B can be either heuristic or learned.

7. Combining Policy and Value

7.1. One-Step Value Reranking

For each legal pick a , construct the hypothetical pool $y^a = y + \mathbf{1}_a$, build a deck $(d^a, b^a, c^a) = B(y^a, m)$, and score

$$S(a | s) = \log \pi_\theta(a | s) + \lambda \text{logit}(V_\psi(d^a, b^a, c^a, r_0)), \quad (24)$$

where r_0 is a reference rank/event context. The recommended pick is $\arg \max_{a \in \mathcal{A}(s)} S(a | s)$. Small λ recovers imitation behavior; larger λ emphasizes outcome optimization.

7.2. Rollout Evaluation

One-step reranking is myopic. A stronger method evaluates each candidate by simulating the remainder of the draft. Let $\text{Rollout}(s, a, \pi_\theta, \eta)$ force action a at s , then complete the draft using policy π_θ for the player and opponent models for other seats, under randomness η . The rollout value is

$$Q(a | s) = \mathbb{E}_\eta [V_\psi(B(Y_T^{s,a,\eta}, m), r_0)], \quad (25)$$

where Y_T is the terminal pool. We combine prior and rollout value:

$$S_R(a | s) = \log \pi_\theta(a | s) + \lambda \text{logit}(Q(a | s)). \quad (26)$$

Algorithm 2 Rollout policy-value decision

Require: State s , simulations R , policy π_θ , value V_ψ , deckbuilder B

- 1: **for** $a \in \mathcal{A}(s)$ **do**
- 2: $q \leftarrow 0$
- 3: **for** $r = 1$ to R **do**
- 4: $Y_T \leftarrow \text{Rollout}(s, a, \pi_\theta, \eta_r)$
- 5: $(d, b, c) \leftarrow B(Y_T, m)$
- 6: $q \leftarrow q + V_\psi(d, b, c, r_0)$
- 7: **end for**
- 8: $Q(a) \leftarrow q/R$
- 9: $S(a) \leftarrow \log \pi_\theta(a | s) + \lambda \text{logit}(Q(a))$
- 10: **end for**
- 11: **return** $\arg \max_a S(a)$

7.3. MCTS with Policy Priors

Monte Carlo tree search uses π_θ as the prior and $V_\psi \circ B$ as the leaf evaluator. At node s , child a is selected by PUCT:

$$a^* = \arg \max_{a \in \mathcal{A}(s)} \left[\bar{Q}(s, a) + c_{\text{puct}} \pi_\theta(a | s) \frac{\sqrt{N(s)}}{1 + N(s, a)} \right]. \quad (27)$$

Leaves are expanded with policy priors. A leaf state is evaluated by rolling out, deckbuilding, and applying V_ψ , or by a learned pool-value approximation. The final decision may use visit counts or posterior action value.

8. Preference Fine-Tuning

The outcome model can create static preference pairs for direct preference optimization. For a logged state s , choose two legal actions a and b . Estimate $Q(a | s)$ and $Q(b | s)$ by one-step scoring or rollouts. If $Q(a) > Q(b) + \epsilon$, create preference $(s, a \succ b)$; if $Q(b) > Q(a) + \epsilon$, create $(s, b \succ a)$. Human disagreement audits can be added to the same preference set.

In addition to deck-quality preferences, we also train a lightweight game-card bonus model from aggregate outcome data. The best variant is a regularized card-main-effect binomial model,

$$\text{logit}(\text{Pr}(\text{win})) = \beta_0 + \sum_i d_i \beta_i + \rho^\top r, \quad (28)$$

where d_i indicates card inclusion and r are covariates. Pair-interaction variants were tested but overfit relative to the card-only model. Game-card preference pairs are then generated by preferring the in-pack candidate with larger learned β_i , subject to a minimum gap. This produces both capped and uncapped DPO datasets; the uncapped dataset contains approximately 1.83M train pairs and 0.20M eval pairs.

Algorithm 3 MCTS pick recommendation

Require: Root state s_0 , simulations M , policy π_θ , value V_ψ , deckbuilder B

- 1: **for** $m = 1$ to M **do**
- 2: $s \leftarrow s_0$; path $\leftarrow []$
- 3: **while** s is expanded and nonterminal **do**
- 4: choose a by PUCT
- 5: append (s, a) to path
- 6: $s \leftarrow \text{Step}(s, a)$, including opponent-policy transitions as needed
- 7: **end while**
- 8: **if** s is terminal **then**
- 9: $v \leftarrow V_\psi(B(Y_T, m), r_0)$
- 10: **else**
- 11: expand s with priors $\pi_\theta(\cdot | s)$
- 12: complete or bootstrap to estimate v
- 13: **end if**
- 14: **for each** (s, a) in path **do**
- 15: update $N(s)$, $N(s, a)$, and $\bar{Q}(s, a)$ with v
- 16: **end for**
- 17: **end for**
- 18: **return** $\arg \max_{a \in \mathcal{A}(s_0)} N(s_0, a)$

Given a frozen reference policy π_{ref} , DPO optimizes

$$\begin{aligned} \Delta_\theta(s, a^+, a^-) &= \log \frac{\pi_\theta(a^+ | s)}{\pi_{\text{ref}}(a^+ | s)} \\ &\quad - \log \frac{\pi_\theta(a^- | s)}{\pi_{\text{ref}}(a^- | s)}, \\ \mathcal{L}_{\text{DPO}}(\theta) &= -\mathbb{E}_{(s, a^+, a^-)} \log \sigma(\tau \Delta_\theta(s, a^+, a^-)). \end{aligned} \quad (29)$$

where τ is an inverse-temperature hyperparameter. This moves the policy toward outcome-preferred picks while anchoring it to the human-imitation model.

9. Evaluation

We evaluate the policy component by top-1 accuracy, top-3 accuracy, and mean reciprocal rank on held-out human picks, with draft-level splits to avoid leakage. We evaluate the value model by held-out log loss, Brier score, calibration error, and rank-stratified AUC. The combined assistant is evaluated by offline counterfactual disagreement audits: positions where the assistant and logged human pick differ are sampled for expert review, and by simulated draft rollouts scored by the frozen value model. Synthetic out-of-vocabulary evaluation with held-out cards measures robustness to cube-list changes. For the kNN fallback, we use leave- N -cards-out bootstrap evaluations and a small curated neighbor sanity set, including examples such as *Figure of Fable* $\rightarrow$ *Figure of Destiny*, *Skrelv, Defector Mite* $\rightarrow$ *Mother of Runes/Giver of Runes*, and *Kellan, the*

Algorithm 4 Training pipeline

- 1: Build $\mathcal{D}_\pi$ from 17lands draft rows by reconstructing pack, pool, and seen-unpicked features per draft.
- 2: Train π_{θ_0} on $\mathcal{D}_\pi$ using $\mathcal{L}_\pi$.
- 3: For deployment cube lists, resolve Arena/Scryfall ids and precompute kNN pseudo-embeddings for cards absent from the training vocabulary.
- 4: Build $\mathcal{D}_V$ from 17lands game rows; aggregate by deck build when useful.
- 5: Train V_ψ using $\mathcal{L}_V$ or $\mathcal{L}_V^{\text{agg}}$.
- 6: Fit or implement deckbuilder B from draft pools to built decks.
- 7: Generate preference pairs with $V_\psi \circ B$ and optional human audits.
- 8: Initialize $\pi_{\text{ref}} \leftarrow \pi_{\theta_0}$ and fine-tune π_θ with $\mathcal{L}_{\text{DPO}}$.
- 9: Deploy with reranking, rollout search, or MCTS depending on latency budget.

Table 1. Game-card bonus outcome model variants. Lower is better.

Variant	Log loss	Brier
Card-only	0.67798	0.05528
1k pairs	0.67943	0.05594
3k pairs	0.68319	0.05763
1k pairs, strong reg.	0.67929	0.05588

Fae-Blooded $\rightarrow$ *Stoneforge Mystic*.

10. Empirical Results

Human-pick imitation and value reranking. Continuing human-pick training from an outcome-weighted checkpoint yields final held-out metrics

$$\text{top1} = 0.618, \quad \text{top3} = 0.908, \quad \text{MRR} = 0.767. \quad (30)$$

On a 20k sampled sweep, the strongest human-like continued model obtains top-1 0.6240, top-3 0.9093, and generated preference accuracy 0.5494. Adding value reranking to this model raises preference accuracy to 0.6062 with nearly unchanged human-pick metrics.

Game-card bonus model. We trained card-only and pair-interaction binomial outcome models on aggregated 17lands Powered Cube outcomes. Table 1 shows that the card-only model gives the best validation loss; pair interactions are noisy in this data regime.

The largest learned bonuses are qualitatively plausible for Powered Cube, including *Ancestral Recall*, *Time Walk*, *Black Lotus*, *Sol Ring*, *Mana Crypt*, and the Moxen.

Table 2. 20k sampled policy sweep. $\lambda = 1$ denotes calibrated value reranking with temperature 1.2016 where applicable.

Config	Top-1	Top-3	Pref. acc.
Human continued, $\lambda = 0$	0.6240	0.9093	0.5494
Human continued, $\lambda = 1$	0.6234	0.9077	0.6062
Safer game-data, $\lambda = 1$	0.6161	0.9048	0.6406
Aggressive pref-acc, $\lambda = 1$	0.5092	0.8411	0.7262

DPO from game-card preferences. Table 2 summarizes the main policy trade-off. The safer uncapped DPO model improves generated game-data preference behavior while retaining high human agreement. The aggressive preference-accuracy model substantially increases preference accuracy, but at a large cost to human-pick imitation. Extending aggressive training from 100k to 500k steps increased training margins but did not materially improve external preference accuracy, so the 100k checkpoint is used as the experimental aggressive preset.

Qualitative behavior. On an actual held-out 17lands draft sequence, the safer model matched the human on picks such as *Skyclave Apparition* and *Windswept Heath*, while the aggressive model selected higher game-bonus alternatives such as *Ouroboroid*, *Birds of Paradise*, *Demonic Tutor*, and *Zuran Orb*. This illustrates the intended behavior of the aggressive model: it is not a human-imitation policy, but a value/preference-seeking policy.

We also evaluated the search layer on real depleted packs from a held-out 17lands draft prefix. In one eight-pick trace, the logged human followed a white/fixing lane with picks such as *Skyclave Apparition*, *Windswept Heath*, *Scrubland*, *Sacred Foundry*, and *Winds of Abandon*. MCTS instead made an early *Blood Crypt* choice and then produced a coherent black-red, reanimator-leaning line: *Hymn to Tourach*, *Demonic Tutor*, *Valki*, *God of Lies*, *Archon of Cruelty*, *Emperor of Bones*, and *Bone Shards*. This trace is not a counterfactual replay of hidden packs, but it suggests that search can improve internal strategic consistency: it may amplify a questionable early lane choice, yet subsequent picks are more coherent with that lane than static one-step policies.

MCTS latency. We benchmarked inference-time search with models loaded once on real held-out pick states. Table 3 reports wall-clock CPU latency. A 25-simulation budget is practical for live use as an optional search-based reranker.

Function-aware unseen-card interpolation. For the Arena Powered Cube 5.0 list published by Wizards, 117 of 540 cards were absent from the current 17lands Powered Cube scoring vocabulary. We resolved all cards using local Arena card databases, Scryfall bulk data, and backup Scryfall lookup. We then precomputed top-8 kNN

Table 3. MCTS inference latency on CPU with checkpoints preloaded.

Simulations	p50	p95	Mean
10	418 ms	574 ms	378 ms
25	942 ms	1370 ms	850 ms
50	1825 ms	2697 ms	1642 ms

interpolation weights for the missing cards and cached them for live inference. A bootstrap synthetic-OOV evaluation over 20k held-out pick states, with 20 cards held out for each of 200 trials, showed small overall degradation because only a minority of states are affected: top-1 -0.0068 , top-3 -0.0036 , and MRR -0.0049 . On directly affected states where the picked card was held out, the average drops were larger: top-1 -0.1253 , top-3 -0.0714 , and MRR -0.0902 , indicating that kNN is a useful fallback but not a replacement for learned embeddings. After tuning the similarity weights on curated functional anchors, the cached neighbors match several important qualitative expectations: *Figure of Fable*→*Figure of Destiny*, *Skrelv*, *Defector Mite*→*Giver of Runes/Mother of Runes*, *Cloud*, *Midgar Mercenary*→*Stoneforge Mystic*, *Kellan*, *the Fae-Blooded*→*Stoneforge Mystic*, and *Sear*→*Lightning Bolt/Abrade*.

Live-client integration. We implemented an experimental MTGA Player.log follower for macOS/Epic Games installs. In a Quick Draft log, the client emitted scene changes into Draft, outgoing BotDraftDraftPick messages, and nested payloads containing DraftPack, PickedCards, PackNumber, and PickNumber. In later Arena draft logs, the client emitted Draft.Notify and EventPlayerDraftMakePick payloads with Arena GrpIds. The assistant maps these ids through Arena’s local Raw SQLite file, strips localization markup such as <nobr>, falls back to Scryfall bulk/API lookup for missing cards, and uses the precomputed kNN cache for cards absent from the scoring vocabulary. This is sufficient to reconstruct live pack and pick state for formats whose card ids can be mapped to names. Non-cube drafts require an up-to-date global Arena id map and are out-of-domain for the Powered Cube policy.

11. Discussion

The system intentionally avoids end-to-end reinforcement learning as a first step. Human pick imitation supplies stable local behavior, game outcomes supply a separate terminal signal, and search or preference fine-tuning combines them without high-variance credit assignment across an entire draft. This decomposition also cleanly separates three sources of error: policy imitation error, deckbuilder error,

and value-model outcome error.

References

17Lands. 17lands public datasets. https://www.17lands.com/public_datasets, 2026. Accessed 2026-06-24.

Scryfall. Scryfall bulk data. <https://scryfall.com/docs/api/bulk-data>, 2026. Accessed 2026-06-24.